



CMPT 295

Unit - Machine-Level Programming

Lecture 10 – Assembly language basics: **move** operation

Last Lecture

► Review: von Neumann architecture

- Data and code are both stored in memory during program execution

1. How our C program/code can be represented as a series of 0's and 1's (i.e., as machine code)

► Compilation process (gcc):

C program -> assembly code -> machine level code -> executable (file)

- Compiler made of tools: 1) C preprocessor, 2) C compiler, 3) assembler, 4) linker

2. Then, how our C program/code, once it is represented as a series of 0's and 1's, ends up being stored in memory

- When C program is executed (tool: loader) (e.g. from our demo: `./ss 5 6`)
loader: loads file `ss` from external storage into memory

3. Then, how our C program (once it is represented as a series of 0's and 1's and it is stored in memory) ends up being executed by the microprocessor (aka the CPU)

- CPU executes C program by looping through the fetch-execute cycle

Last Lecture

- As **x86-64** assembly s/w dev., we now get to see more of the microprocessor (CPU): PC, registers, condition codes
- **x86-64** assembly language – **Data**
 - 16 integer registers of 64 bits each: can manipulate either 1, 2, 4 or 8 bytes (memory address of 8 bytes as well) depending on the register name used
 - Floating point registers of 64 bits each: can manipulate either 4 or 8 bytes depending on the register name used
 - No compound data types such as arrays or structures

Today's Menu

- Introduction
 - C program -> assembly code -> machine level code
- Assembly language basics: data, `move` operation
 - Memory addressing modes
- Operation `leaq` and Arithmetic & logical operations
- Conditional Statement – Condition Code + `cmovX`
- Loops
- Function call – Stack
 - Overview of Function Call
 - Memory Layout and Stack - x86-64 instructions and registers
 - Passing control
 - Passing data – Calling Conventions
 - Managing local data
 - Recursion
- Array
- Buffer Overflow
- Floating-point data & operations

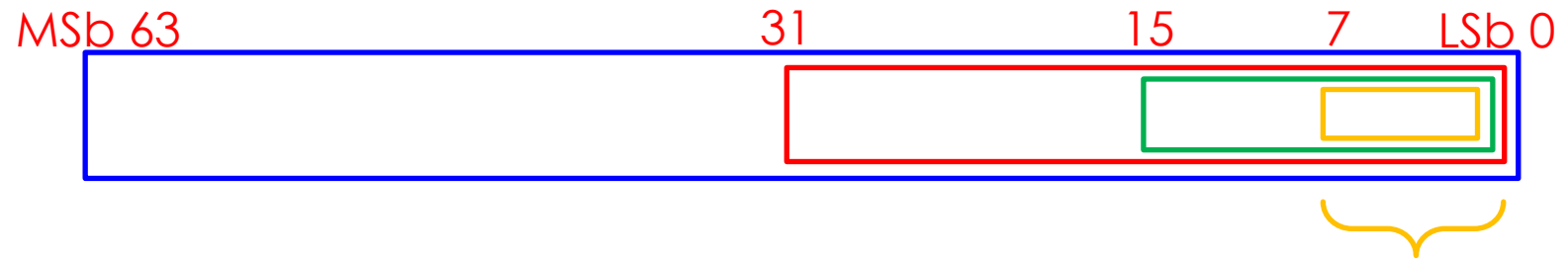
x86-64 Assembly Language – Data

Integer Registers

64-bit (quad)	32-bit (double)	16-bit (word)	8-bit (byte)	
63..0	31..0	15..0		7..0
rax	eax	ax		al
rbx	ebx	bx		bl
rcx	ecx	cx		cl
rdx	edx	dx		dl
rsi	esi	si		sil
rdi	edi	di		dil
rbp	ebp	bp		bpl
rsp	esp	sp		spl
r8	r8d	r8w		r8b
r9	r9d	r9w		r9b
r10	r10d	r10w		r10b
r11	r11d	r11w		r11b
r12	r12d	r12w		r12b
r13	r13d	r13w		r13b
r14	r14d	r14w		r14b
r15	r15d	r15w		r15b

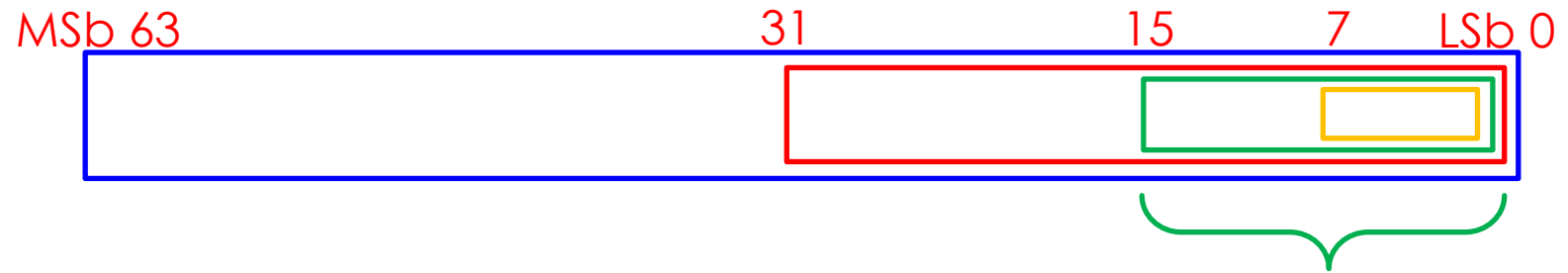
- Storage locations in CPU
-> fastest storage
- 16 registers are used explicitly – must name them in assembly code
- Some registers are used implicitly
 - e.g., PC, FLAGS
- Each register is 64 bits in size, but we can refer to its:
 - first byte LSB (8 bits),
 - first 2 bytes (16 bits),
 - first 4 bytes (32 bits),
 - or to all of its 8 bytes (64 bits)

About these integer registers!



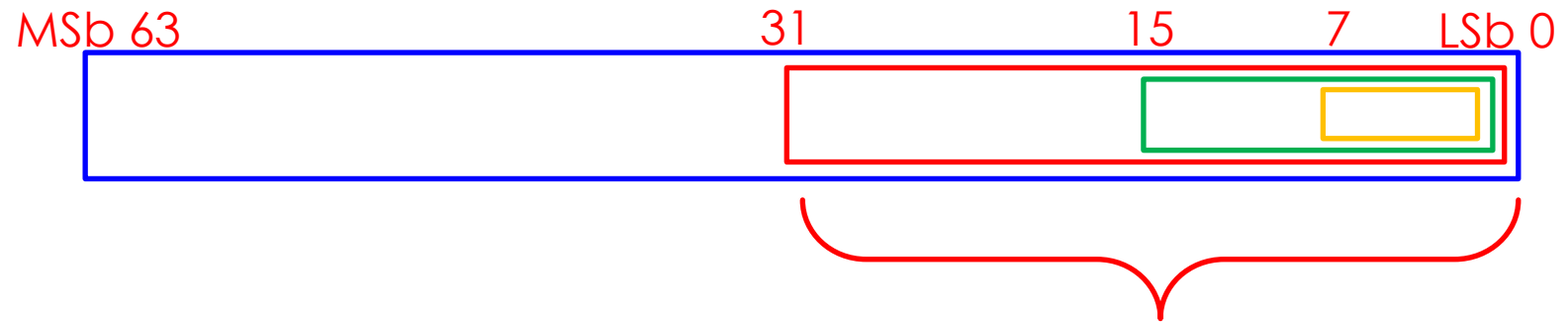
If I want 8 bits worth of data, then I can use register names such as `%a1` or `%di1` or `%r12b`

About these integer registers!



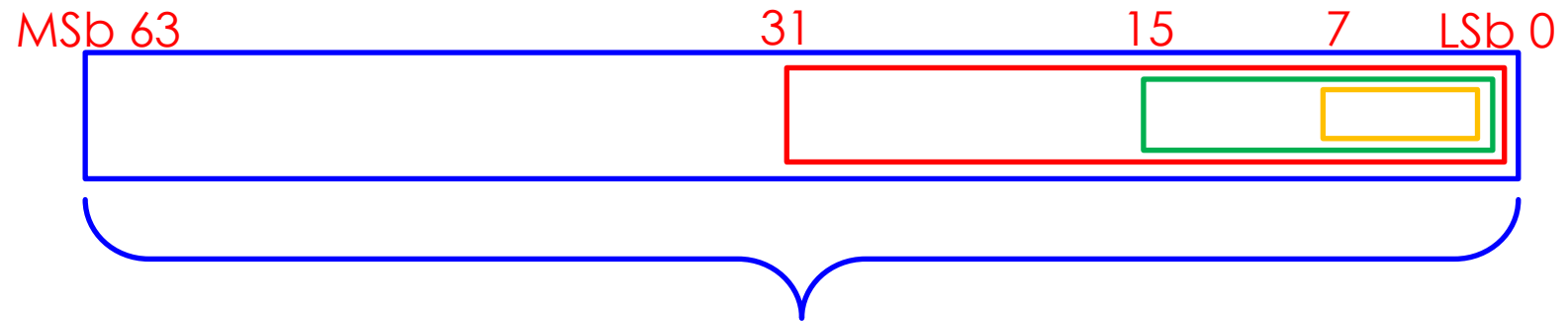
If I want **16** bits worth of data, then I can use register names such as **%ax** or **%di** or **%r12w**

About these integer registers!



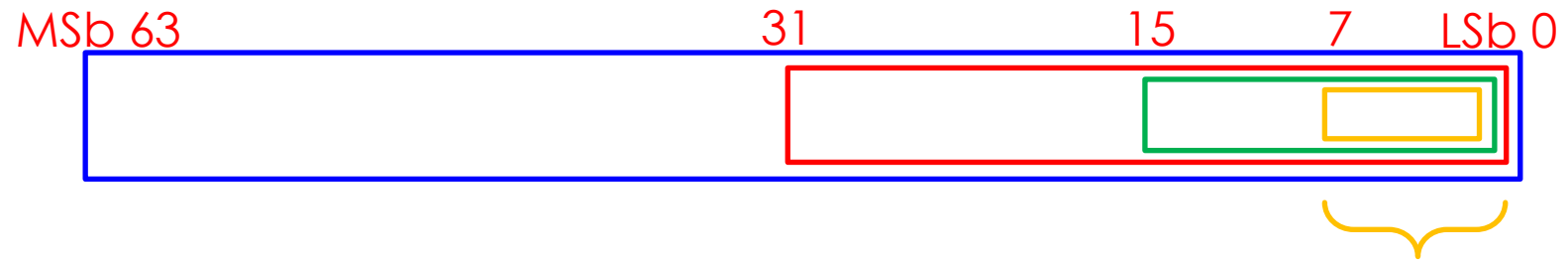
If I want **32** bits worth of data, then I can use register names such as **%eax** or **%edi** or **%r12d**

About these integer registers!



If I want **64** bits worth of data, then I can use register names such as **%rax** or **%rdi** or **%r12**

About these integer registers!



If I want **8** bits worth of data, then I can use register names such as **%a1** or **%di1** or **%r12b**

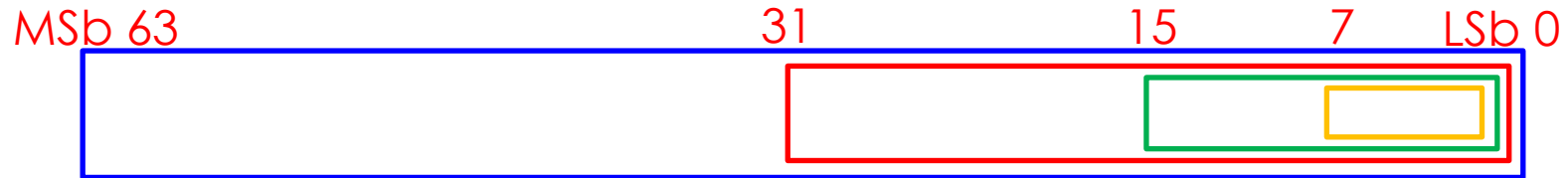
If I want **16** bits worth of data, then I can use register names such as **%ax** or **%di** or **%r12w**

If I want **32** bits worth of data, then I can use register names such as **%eax** or **%edi** or **%r12d**

If I want **64** bits worth of data, then I can use register names such as **%rax** or **%rdi** or **%r12**

Remember that for all 16 registers ...

Let's use the register associated with the names `%rax`, `%eax`, `%ax` and `%al` as an example:



- `%rax`, `%eax`, `%ax` and `%al` all refer to the **same register**
- However...
 - Each refer to a different section of this register
 - `%rax` refers to all 64 bits of this register
 - `%eax` refers to only 32 bits of this register
 - the LS 32 bits of it -> bit 0 to bit 31
 - `%ax` refers to only 16 bits of this register
 - the LS 16 bits of it -> bit 0 to bit 15
 - `%al` refers to only 8 bits of this register
 - the LS 8 bits of it -> bit 0 to bit 7

x86-64 Assembly Language - Instructions

- “2 operand” assembly language
- x86-64 functionally complete -> i.e., it is **Turing complete**
 - 3 classes of instructions
 1. **Memory reference => Data transfer instructions**
 - Transfer data between memory and registers
 - **Load** data from memory into register
 - **Store** register data into memory
 - **Move** data from one register to another
 2. **Arithmetic and logical => Data manipulation instructions**
 - Perform calculations on register data
 - e.g., arithmetic, logic, shift
 3. **Branch and jump => Program control instructions**
 - Transfer control
 - Unconditional jumps to/from functions
 - Unconditional/conditional branches

Move data – **mov***

1. **Memory reference => Data transfer instructions**

➤ Transfer data between memory and registers

➤ Syntax: **mov*** *Source*, *Destination*

➤ Example: **movq** %rdi, %rax

➤ Allowed moves:

- From register to register (**move**)
- From memory to register (**load**)
- From register to memory (**store**)

➤ Conditional move (**cmov***)

- Same as above, but based on result of comparison

*** -> Size designator**

q -> long 64

l -> int 32

w -> short 16

b -> char 8

Demo – Swap Function

- Problem: Let's swap the contents of two variables
- For now, we need to know that
 - Argument 1 of function `swap(...)` -> saved in `%rdi`
 - Argument 2 of function `swap(...)` -> saved in `%rsi`

Demo – Swap Function

```
void swap(long *xp, long *yp)
{
    long L1 = *xp;
    long L2 = *yp;
    *xp = L2;
    *yp = L1;
    return;
}
```

```
swap:
    movq    (%rdi), %rax    # L1 = *xp
    movq    (%rsi), %rdx    # L2 = *yp
    movq    %rdx, (%rdi)    # *xp = L2
    movq    %rax, (%rsi)    # *yp = L1
    ret
```

Registers

%rdi	
%rsi	
%rax	
%rdx	

Memory => Remember:

Address

0x0020	
0x0018	
0x0010	
0x0008	
0x0000	

*Compressed
view of memory*

Operand Combinations for `mov*`

Source		Dest	Src, Dest	in C
Memory addressing modes				
<code>mov*</code>	Immediate	Register	<code>movq \$0x4, %rax</code>	<code>result = 0x4;</code>
		Memory	<code>movq \$-147, (%rax)</code>	<code>*result = -147;</code>
	Register	Register	<code>movq %rax, %rdx</code>	<code>var1 = result;</code>
		Memory	<code>movq %rax, (%rdx)</code>	<code>*var1 = result;</code>
	Memory	Register	<code>movq (%rax), %rdx</code>	<code>var1 = *result;</code>

Cannot do memory-memory transfer with a single `mov*` instruction

Homework

- ▶ Since we cannot do memory-memory transfer with a single `mov*` instruction ...
 - ▶ Can you write a little **x86-64** assembly program that transfers data stored at address `0x0000` to address `0x0018` ?

Registers

%rdi	
%rsi	
%rax	
%rdx	

Memory

Address	
0x0020	
0x0018	
0x0010	
0x0008	
0x0000	6

Summary

- **x86-64 assembly language – Data**
 - 16 integer registers of 64 bits each: can manipulate either 1, 2, 4 or 8 bytes (memory address of 8 bytes as well) depending on the register name used
 - Floating point registers of 64 bits each: can manipulate either 4 or 8 bytes depending on the register name used
 - No compound data types such as arrays or structures
- **x86-64 assembly language – Instructions**
 - **mov*** instruction family
 - From register to register
 - From memory to register
 - From register to memory
 - *Cannot do memory-memory transfer with a single `mov*` instruction*
 - **Memory addressing modes**

Next Lecture

- Introduction
 - C program -> assembly code -> machine level code
- Assembly language basics: data, `move` operation
 - Memory addressing modes
- Operation `leaq` and Arithmetic & logical operations
- Conditional Statement – Condition Code + `cmovX`
- Loops
- Function call – Stack
 - Overview of Function Call
 - Memory Layout and Stack - x86-64 instructions and registers
 - Passing control
 - Passing data – Calling Conventions
 - Managing local data
 - Recursion
- Array
- Buffer Overflow
- Floating-point data & operations